FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as “the unacknowledged use, as one’s own work, of work of another person, whether or not such work has been published” (<u>Regulations Governing Conduct at Examinations</u>, 1997, Regulation 1 (viii), University of Malta).

I / We*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our* work, except where acknowledged and referenced.

I / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

David Farrugia
Student Name

Signature

Student Name

Signature

Student Name

Signature

Student Name

Signature

ARI5121
Course Code

ARI5121_SpeechProject_Farrugia_David.zip
Title of work submitted

22/05/2026
Date

Speaker Voice Similarity Analysis and Evaluation

David Farrugia (0015004H)
david.farrugia.22@um.edu.mt
M.Sc. In Artificial Intelligence
University of Malta
Msida, Malta

Abstract

This report evaluates the use of a pre-trained WavLM speaker verification model for speaker voice similarity analysis on `shortpassage` WAV files from the ABI-1 Corpus. Speaker embeddings were extracted from each recording and compared using cosine similarity with a threshold-based decision rule. The implementation was evaluated using classification metrics, confusion matrices and PCA and t-SNE visualisations. The results show that a stricter threshold of 0.98 produced the strongest performance by reducing false positives while maintaining high recall.

1 Introduction

This project focuses on speaker voice similarity, which is closely related to speaker identification and speaker verification. Speaker identification aims to determine who is speaking from an audio recording, while speaker verification checks whether two speech samples are likely to come from the same speaker. These tasks are possible because speech contains speaker-dependent characteristics such as vocal tract properties, speaking style, rhythm and other vocal cues (Reynolds, 2002). In speaker recognition systems, these characteristics are often represented through fixed-length speaker embeddings, which allow speech recordings of different lengths to be compared in a common vector space (Snyder et al., 2018).

A common challenge in speech processing is that useful representations need to capture information from raw audio without relying only on manually designed features. Models such as wav2vec 2.0 showed that speech representations can be learned directly from raw audio using large amounts of unlabelled speech data, before being adapted to specific use cases (Baevski et al., 2020). WavLM builds on this by learning general-purpose speech representations through masked speech prediction and speech denoising, making it suitable for a range of speech processing tasks, including speaker-related applications such as voice biometrics and speaker diarization (Chen et al., 2022).

In this project, the pre-trained `microsoft/wavlm-base-plus-sv` is used to extract speaker embeddings for speaker voice similarity analysis from speech samples taken from the Accents of the British Isles (ABI-1) Corpus, which contains recordings from speakers across several regional accent groups (Najafian and Russell, 2020). Each selected audio file is converted into a fixed-length embedding and pairs of embeddings are compared using cosine similarity. A threshold-based decision rule is then applied, where pairs with similarity scores above the selected threshold are classified as same-speaker pairs, while others are classified as different-speaker pairs.

The main goal of this project is to assess how effectively WavLM speaker embeddings can distinguish between same-speaker and different-speaker audio pairs in the ABI-1 Corpus. The system is evaluated quantitatively using threshold testing, accuracy, precision, recall, specificity, F1 score and confusion matrices. Additionally, PCA and t-SNE plots are used to inspect the structure of the embedding space and assess if samples from the same speakers form meaningful clusters (Jolliffe and Cadima, 2016; van der Maaten and Hinton, 2008).

2 Data Preprocessing

The dataset used in this project is the Accents of the British Isles (ABI-1) Corpus, which contains recordings from 285 speakers across several regional accent groups. The corpus is organised into accent folders, with speaker data further separated by gender and speaker ID. Although each speaker folder contains audio recordings (.WAV), transcription files (.TRS) and text files (.TXT), this project only makes use of the WAV files containing `shortpassage` in the filename. These recordings are approximately 40-60 seconds long and contain read speech passages (Najafian and Russell, 2020; D'Arcy and Russell, 2008).

The data preprocessing stage began by filtering the original ABI-1 directory to keep only the required `shortpassage` WAV files. Transcription files were excluded because the experiment focuses on speaker voice similarity rather than speech recognition or transcript-based analysis. The selected WAV files were then copied into a separate cleaned directory while preserving the original folder structure. This kept the implementation focused only on the relevant audio files, while still allowing useful metadata, including accent, gender and speaker ID, to be extracted from the file path and stored in a metadata CSV file.

Each row in the metadata file stores the accent, gender, speaker ID and full path for one selected audio file. This file was used as the index for the embedding extraction stage, to make sure that each recording could be linked back to its speaker label during pairwise comparison and evaluation.

For model input, each WAV file was loaded using `librosa`, converted to mono and resampled to 16 kHz. This sample rate was selected because it matches the expected input format of the `microsoft/wavlm-base-plus-sv` model (Microsoft, 2021). After loading and resampling, the waveform was then passed to the WavLM feature extractor, which converts the raw audio into the tensor format expected by the pre-trained model.

3 Methodology

This methodology section discusses the implementation of a speaker voice similarity pipeline using Microsoft's WaveLM model. The implementation itself, was carried out in a Linux environment using WSL and `Python 3.12.3`. For reproducibility, the full code for the pipeline is available in a public GitHub repository[1].

3.1 Speaker Embedding Extraction

The first stage involved extracting speaker embeddings from the cleaned ABI-1 audio files. The pre-trained `microsoft/wavlm-base-plus-sv` model was used for this task, since WaveLM is designed to learn general-purpose speech representations (Chen et al., 2022), while this specific Hugging Face model is intended for speaker verification (Microsoft, 2021). The model and its feature extractor were then loaded using the Hugging Face `transformers` library, while `torch` was used to run model inference.

Each selected WAV file was loaded using `librosa`, converted to mono and resampled to 16 kHz. This was done to match the expected input format of the WavLM speaker verification model (Microsoft, 2021). The waveform was then passed through the feature extractor, which converted the audio signal into the tensor format required by the model. Since the purpose of this project was evaluation, the model was placed in evaluation mode and embeddings were extracted without calculating gradients.

For each recording, the model produced a fixed-length speaker embedding. These embeddings are commonly used in speaker recognition since they allow speech samples to be represented as numerical vectors that can later be compared (Snyder et al., 2018). After this, each embedding was stored together with its corresponding metadata, including the accent, gender, speaker ID and file path. Keeping this metadata attached to each embedding is important as the speaker ID was later used to determine whether a pair of recordings belonged to the same speaker or to different speakers.

Additionally, to make the pipeline more efficient, the extracted embeddings were saved to a `.pkl` file. This allowed later stages of the implementation, such as pairwise comparison and threshold evaluation, to be repeated and tested without having to extract the embeddings again as it was a time consuming process.

3.2 Pairwise Cosine Similarity Comparison

After extracting the speaker embeddings, the next stage involved comparing them using cosine similarity. Every unique pair of embeddings was generated using the `combinations` function from Python's `itertools` library. This ensured that each pair was compared only once, while avoiding duplicate comparisons and cases where an embedding was compared with itself. A cosine similarity score was then calculated for each pair of audio samples.

Cosine similarity was used because it measures

[1] https://github.com/DavidF-22/ARI5121-AppliedNLP_Project/tree/main/SpeechProcessing

the similarity between two vectors based on their direction in the embedding space. In this project, a higher cosine similarity score indicates that two speech samples have more similar speaker embeddings, while a lower score suggests that they are less likely to come from the same speaker (Reynolds, 2002).

For each pairwise comparison, the system stored the two file paths, the two speaker IDs, the cosine similarity score and a ground-truth label. This label was created by comparing the speaker IDs extracted from the dataset structure. If both recordings had the same speaker ID, the pair was labelled as a same-speaker pair. If the speaker IDs were different, the pair was labelled as a different-speaker pair.

The resulting pairwise similarity table formed the basis of the evaluation stage. It allowed the system to test how well different cosine similarity thresholds separated same-speaker pairs from different-speaker pairs.

3.3 Threshold Evaluation and Embedding Visualisation

The final stage of this methodology, evaluates the similarity scores using a threshold-based decision rule. A pair was classified as same-speaker if its cosine similarity score was greater than or equal to the selected threshold. Otherwise, it was classified as a different-speaker pair. Several threshold values were tested (`np.arange(0.50, 0.96, 0.05)`) and (`np.arange(0.90, 1.001, 0.01)`) to test how the performance changed as the decision boundary became stricter. The two best thresholds obtained are `0.95` and `0.98` respectively. This follows the speaker verification approach, where similarity scores are used to decide whether two speech samples belong to the same speaker or to different speakers (Reynolds, 2002).

For each threshold, the predicted labels were compared against the ground-truth labels created during the comparison stage. Accuracy was calculated for each threshold and the best threshold was selected based on the highest accuracy. The final selected threshold was then used to calculate accuracy, precision, recall, specificity and F1 score. Accuracy measures overall correct classifications, while precision and recall show how reliable the predictions were and how many true same-speaker pairs were successfully identified. Moreover, specificity was also included because the comparison process creates many more different-speaker pairs than same-speaker pairs, This made it important to check how well the system rejected different-speaker comparisons, rather than only relying only on accuracy.

In addition to these metrics, dimensionality reduction was also done to inspect the speaker embedding space. PCA and t-SNE were applied to a subset of embeddings of around 20 speakers. PCA was used to provide a simple two-dimensional overview of the speaker embedding space (Jolliffe and Cadima, 2016), while t-SNE was used since it can show local grouping patterns more clearly, helping to check whether recordings from the same speaker appeared close together (van der Maaten and Hinton, 2008). The purpose of these plots was to provide a qualitative view of whether recordings from the same speaker formed visible clusters, this supported the idea that the WavLM embeddings captured speaker-specific information (Chen et al., 2022; Snyder et al., 2018).

4 Evaluation and Results

This section highlights the results obtained from this project implementation. Since threshold selection was carried out in two stages, the evaluation first compares the best-performing thresholds obtained from each search range, which are `0.95` and `0.98`.The evaluation stage then examines the classification performance of both thresholds, followed by the visualisations of the embedding space.

4.1 Threshold Selection

Threshold testing was carried out in two stages. The first stage used a wider search range, `np.arange(0.50, 0.96, 0.05)`, to identify the general region where performance was strongest. This search produced a best threshold of `0.95`. Since this value occurred at the upper end of the tested range, a second stricter search was carried out using `np.arange(0.90, 1.001, 0.01)`. This search identified `0.98` as being the best-performing threshold.

This approach follows the speaker verification setting, where similarity scores are converted into same-speaker or different-speaker decisions using a decision threshold (Reynolds, 2002). Figure 1 shows the accuracy curve for the wider search, while Figure 2 shows the stricter search space. The first search was useful for finding the high-performing region, while the second search allowed the decision boundary to be tested more precisely.

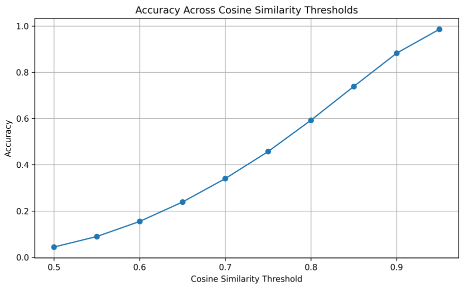


Figure 1: Accuracy across thresholds for the wider search.

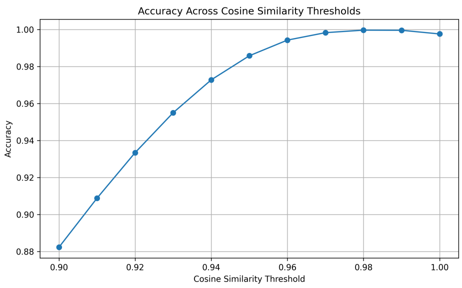


Figure 2: Accuracy across thresholds for the stricter search.

Moreover, Table 1 shows that although the threshold of `0.95` achieved a high accuracy of `0.985836`, its precision was only `0.142097`. This means that many pairs predicted as same-speaker were actually different-speaker pairs. On the other hand, with a threshold of `0.98` improved precision to `0.895570` while still maintaining a high recall of `0.982639`. It also increased the F1 score from `0.248510` to `0.937086`, showing a stronger balance between correctly identifying same-speaker pairs and avoiding false positives. Due to this, `0.98` was selected as the final threshold for this project.

Metrics	**0.95**	**0.98**
`Accuracy`	0.985836	0.999688
`Precision`	0.142097	0.895570
`Recall`	0.989583	0.982639
`Specificity`	0.985827	0.999728
`F1 score`	0.248510	0.937086

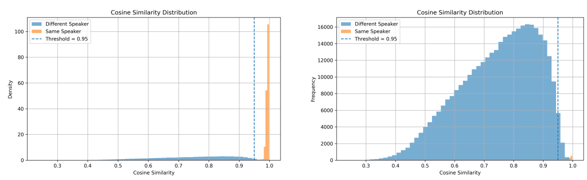
Table 1: Comparison of the two threshold search stages.

4.2 Cosine Similarity Distributions

To better understand the effect of the two thresholds, the cosine similarity score distributions were examined for same-speaker and different-speaker pairs. Both density and frequency plots were plotted as they highlight different aspects of the results. The density plots make it easier to compare the overall shape of the same-speaker and different-speaker score distributions, while the frequency plots show the actual number of comparisons in each score range. This is important since the comparison process produced many more different-speaker pairs than same-speaker pairs.

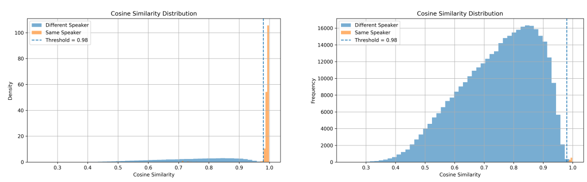
Figure 3 shows the cosine similarity distributions at threshold `0.95`, while Figure 4 shows the distributions at threshold `0.98`. In both threshold settings, same-speaker pairs generally receive higher cosine similarity scores than different-speaker pairs. This suggests that the embeddings capture useful speaker-related information, since recordings from the same speaker tend to be mapped closer together in the embedding space (Snyder et al., 2018; Chen et al., 2022). It is also worth noting that both sets of plots show the strong imbalance between the two classes, with different-speaker comparisons occurring much more often.

Moreover, the threshold of `0.95` is less strict and therefore can allow more different-speaker pairs to be classified as same-speaker. This can be the reason for its low precision, since many predicted same-speaker matches.



(a) Density distribution **(b)** Frequency distribution

Figure 3: Cosine similarity score distributions at threshold `0.95`.



(a) Density distribution **(b)** Frequency distribution

Figure 4: Cosine similarity score distributions at threshold `0.98`.

4.3 Classification Performance

The classification behaviour of both thresholds was further evaluated using normalised confusion matrices. These matrices show the proportion of correct and incorrect predictions within each true class, making them more suitable than raw counts when in this case, the number of different-speaker pairs is much larger than the number of same-speaker pairs.

Figure 5 shows the normalised confusion matrix for the threshold `0.95`, while Figure 6 shows the result for `0.98`. The threshold of `0.95` correctly identifies most same-speaker pairs, but it also produces more false positives. In contrast, the normalised confusion matrix for `0.98` shows that fewer different-speaker pairs are incorrectly classified as same-speaker, while most same-speaker pairs are still correctly identified, further supporting the selection of `0.98` as the final threshold, since it reduces false positives without heavily reducing the system's ability to detect true same-speaker pairs.

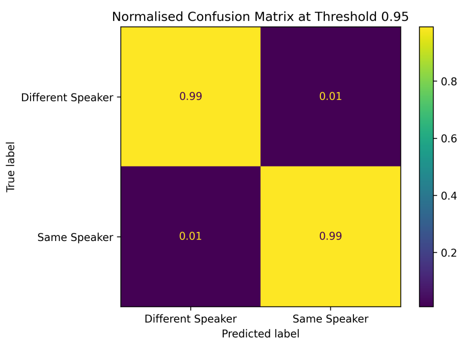


Figure 5: Normalised confusion matrix at threshold `0.95`.

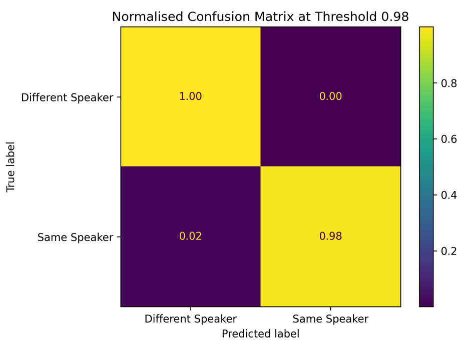


Figure 6: Normalised confusion matrix at threshold `0.98`.

4.4 Embedding Space Visualisation

In addition to the quantitative evaluation, the speaker embedding space was evaluated visually as well using PCA and t-SNE. PCA was used to provide a simple two-dimensional overview of the embedding space (Jolliffe and Cadima, 2016), while t-SNE was used because it can show local grouping patterns in high-dimensional data more clearly (van der Maaten and Hinton, 2008). Since these methods are applied directly to the embeddings rather than to the threshold-based predictions, the visualisations do not depend strongly on whether the selected threshold is `0.95` or `0.98`.

Figure 7 shows the PCA projection of embeddings from around 20 selected speakers, while Figure 8 shows the t-SNE projection. Some recordings from the same speaker appear to cluster together, further suggesting that the WavLM embeddings capture speaker-specific information. This is consistent with the use of neural speaker embeddings in speaker recognition systems (Snyder et al., 2018) and with WavLM's ability to learn general-purpose speech representations (Chen et al., 2022).

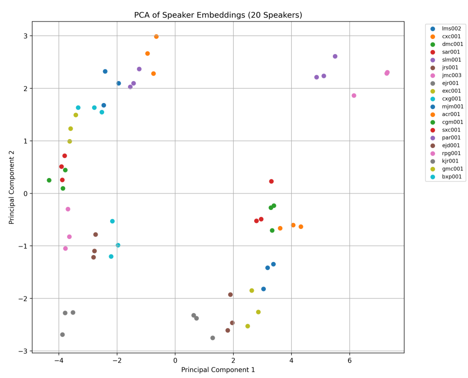


Figure 7: PCA visualisation of speaker embeddings for around 20 selected speakers.

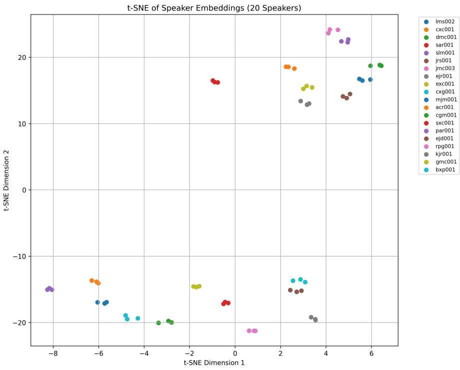


Figure 8: t-SNE visualisation of speaker embeddings for around 20 selected speakers.

5 Analysis and Discussion

The results obtained show that the WavLM speaker embeddings performed strongly for the speaker voice similarity task. The refined threshold of `0.98` achieved the best overall performance, with higher accuracy, specificity and F1 score than the threshold of `0.95`. Although `0.95` achieved high accuracy, its low precision showed that many pairs predicted were actually false positives. At the same time, `0.98` reduced these false positives while still keeping a high recall score, making it the better threshold for this experiment.

This performance is likely linked to the use of `microsoft/wavlm-base-plus-sv`, which is a pre-trained WavLM model intended for speaker verification and because of this, it was expected to separate many same-speaker and different-speaker pairs effectively. This assumption was also supported by the PCA and t-SNE plots, where some recordings from the same speaker appeared clustered in the embedding space.

Regardless, some important limitations still remain. In this implementation the same set of pairwise comparisons was used to select the threshold and report the final results. This means that the selected threshold may be closely fitted to this dataset. An alternative route would be to separate the dataset into a validation set for threshold selection and a held-out test set for final evaluation. This would show more clearly whether the threshold generalises to unseen speakers or other speech datasets.

Another limitation is the imbalance between same-speaker and different-speaker pairs. Since each sample was compared with many other samples, there were many more different-speaker comparisons than same-speaker comparisons. This can make accuracy appear very high even when mistakes occur in the smaller class. For this reason, precision, recall, specificity and F1 score were more useful than accuracy alone.

6 Conclusion and Future Work

This project evaluated the use of WavLM speaker embeddings for speaker voice similarity analysis on the ABI-1 Corpus. The pipeline extracted fixed-length speaker embeddings from selected `shortpassage` WAV files and compared them using cosine similarity with a threshold-based decision rule to classify each pair as either same-speaker or different-speaker.

The results showed that the refined threshold of `0.98` performed better than `0.95`. Although `0.95` achieved high accuracy, it produced more false positives and had low precision. In contrast, `0.98` achieved a stronger performance between precision, recall, specificity and F1 score. This suggests that WavLM embeddings are effective for this speaker similarity task when an appropriate decision threshold is selected.

However, it is worth highlighting that the same pairwise comparisons were used for both threshold selection and final evaluation. This means that the selected threshold may be closely fitted to this dataset. Future work should include splitting the dataset by speaker, using one group for threshold selection and a separate held-out group for final testing. This would provide a fairer estimate of how well the system generalises to unseen speakers.

Further improvements could also include testing the pipeline on additional speech datasets. This could include noisy recordings, shorter speech segments, different microphones, different tones and rhythms or overlapping speakers. Future analysis could also compare performance across accent and gender groups to better understand whether the model performs consistently across different speaker characteristics.

Acknowledgments

This individual project was carried out as part of the partial fulfilment of the requirements for the *ARI5121 Applied Natural Language Processing* course.

References

Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. 2020. wav2vec 2.0: A framework for self-supervised learning of speech representations.

Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, Jian Wu, Long Zhou, Shuo Ren, Yanmin Qian, Yao Qian, Jian Wu, Michael Zeng, Xiangzhan Yu, and Furu Wei. 2022. Wavlm: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):15051518.

Shona D'Arcy and Martin J. Russell. 2008. Experiments with the abi (accents of the british isles) speech corpus. In *Interspeech*.

Ian T Jolliffe and Jorge Cadima. 2016. Principal component analysis: a review and recent developments. *Philos Trans A Math Phys Eng Sci*, 374(2065):20150202.

Microsoft. 2021. microsoft/wavlm-base-plus-sv ů Hugging Face - huggingface.co. `https://huggingface.co/microsoft/wavlm-base-plus-sv`. Accessed: 19 May 2026.

Maryam Najafian and Martin Russell. 2020. Automatic accent identification as an analytical tool for accent robust automatic speech recognition. *Speech Communication*, 122:44–55.

Douglas A. Reynolds. 2002. An overview of automatic speaker recognition technology. In *2002 IEEE International Conference on Acoustics, Speech, and Signal Processing*, volume 4, pages IV–4072–IV–4075.

David Snyder, Daniel Garcia-Romero, Gregory Sell, Daniel Povey, and Sanjeev Khudanpur. 2018. X-vectors: Robust dnn embeddings for speaker recognition. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, page 53295333. IEEE Press.

Laurens van der Maaten and Geoffrey Hinton. 2008. Viualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605.

A Use of GenAI

Generative AI (GenAI) tools were used throughout this project, mainly for research guidance, debugging, understanding speech processing concepts, checking project scope and refining the report. The main tools used were ChatGPT and NotebookLM.

During the early stages, GenAI tools helped clarify concepts such as speaker identification, speaker verification, speaker embeddings, cosine similarity and threshold-based classification. They were also used to understand how a pre-trained WavLM speaker verification model could extract embeddings from audio files and how these embeddings could be compared for same-speaker and different-speaker classification.

GenAI tools were also used during implementation to support debugging and code review, particularly for file path handling, dataset filtering, embedding extraction, result storage and evaluation logic. In the report-writing stage, they were used to review wording, identify unclear sections and check whether the report stayed within the scope of this assignment.

A.1 Prompts and Responses

Prompt 1: *"Can you explain speaker embeddings and cosine similarity in simple terms and how they are used together in speaker verification?"*
Summarised Response: The generated response explained that speaker embeddings are numerical representations of audio recordings that aim to capture speaker-related characteristics. It also explained that cosine similarity measures how close two embeddings are in direction, with higher similarity suggesting that two recordings are more likely to come from the same speaker while also providing sources to blogs and papers.

Prompt 2: *"Does this result mean the model is overfitting? The best threshold is 0.98 and the best accuracy is 1.000." (plots were attached)*
Summarised Response: The AI tool explained that a perfect accuracy score does not necessarily indicate overfitting, but should still be interpreted carefully. It recommended reviewing whether the evaluation setup was too simple, whether speakers or recordings overlapped across comparison sets and whether the test pairs were sufficiently representative. This helped guide the interpretation of the threshold-based results and identify a possible limitation to discuss in the report.

Prompt 3: *"Based on the project description and requirements, is the following draft section in scope? (project description and a draft file where attached)"*
Summarised Response: The tool identified areas that were well aligned with the project scope and areas that needed some improvement. The tools also provided different variations of how the draft can be improved. For example, to include and

expand more on the assumptions made during implementation.

Prompt 4: *"My threshold values are currently being hardcoded with only a four options. How can I test a wider range of possible similarity thresholds for cosine similarity? (Python code bock was attached)"*
Summarised Response: The AI tool explained that testing only a small number of hardcoded thresholds could limit the evaluation, as the most suitable decision boundary might fall between those values. It suggested generating a wider threshold range programmatically through the numpy library, for example using `np.arange()`, to evaluate several possible cosine similarity thresholds. This helped improve the threshold-testing stage and made the final threshold selection more robust.

A.2 Reflection

As a student who prefers hands-on work, I often find theory-heavy components such as report writing and research less engaging, which can lead to them being postponed. However, the use of GenAI tools helped change this approach.

These tools made the theoretical aspects more approachable, particularly during the early stages of background research and identifying relevant references. Tasks such as summarising papers, clarifying concepts and refining the structure of the report became more manageable, which encouraged more consistent engagement with the material.

Furthermore, GenAI also improved productivity by reducing time spent on repetitive tasks, allowing more focus on implementation and experimentation.

Finally, it is worth noting that throughout the project, a responsible approach was maintained. All GenAI outputs were treated strictly as guidance and any suggested material was manually reviewed to ensure accuracy and relevance before being included.